



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

Anna Clara Olbi - 24015392
Bernardo Alberto Amaro - 25014832
Larissa Furlan Ferreira - 24021876
Leonardo Mateus O. Vicente - 24007335
Maria Gabriella Xavier Puccinelli - 24002540
Thiago Rubim Tranquilim - 23015590

Padrões e Arquitetura de Software

CAMPINAS

2026

Introdução

O objetivo do Sprint 0 foi compreender o funcionamento do sistema legado e criar uma base segura para futuras refatorações. Para isso, foram desenvolvidos testes de caracterização utilizando a abordagem Golden Master, cobrindo funcionalidades como criação de pedidos, pagamentos, atualização de status, estoque e relatórios. Os testes permitiram preservar o comportamento original da aplicação antes do início das alterações arquiteturais. Além disso, foi obtida cobertura superior a 85% utilizando a ferramenta coverage.py.

Durante a análise do sistema, foram identificadas diversas violações dos princípios SOLID, evidenciando problemas de acoplamento, manutenção e extensibilidade.

Violações SOLID

SRP — Single Responsibility Principle

A classe Sis concentra múltiplas responsabilidades, incluindo regras de negócio, persistência, pagamentos, notificações e geração de relatórios.

```
self.db = sqlite3.connect('loja.db')  
self.c = self.db.cursor()
```

Além disso:

```
print(f"Email enviado para {n}: Pedido recebido!")
```

Esse modelo dificulta a manutenção e evolução do sistema.

OCP — Open/Closed Principle

O sistema depende de estruturas condicionais extensas para aplicar descontos e processar pagamentos.

```
elif i['tipo'] == 'desc20':  
    tot += i['p'] * i['q'] * 0.8
```

E também:

```
elif m == 'pix':  
    print("Gerando QR Code PIX...")
```

Sempre que um novo comportamento é adicionado, a classe principal precisa ser modificada.

LSP — Liskov Substitution Principle

A classe PedEspecial altera o comportamento esperado da classe base Sis.

```
class PedEspecial(Sis):
```

Além disso:

```
tot = tot * 1.15
```

A subclasse modifica regras da superclasse, podendo gerar inconsistências no sistema.

ISP — Interface Segregation Principle

A classe Sis possui muitos métodos e responsabilidades diferentes, como:

- `add_ped()`
- `proc_pag()`
- `gerar_rel()`
- `validar_estoque()`

Isso aumenta o acoplamento e dificulta a reutilização.

DIP — Dependency Inversion Principle

A aplicação depende diretamente de implementações concretas, como SQLite e `print()`.

```
self.db = sqlite3.connect('loja.db')  
print(f"Email enviado para {n}: Pedido recebido!")
```

Esse acoplamento reduz a flexibilidade e dificulta testes isolados.

Conclusão

A análise do Sprint 0 permitiu identificar problemas arquiteturais importantes no sistema legado, principalmente relacionados à acoplamento excessivo e concentração de responsabilidades.

Os testes Golden Master desenvolvidos garantem maior segurança para futuras refatorações, preservando o comportamento original da aplicação e fornecendo uma base sólida para os próximos sprints do projeto.